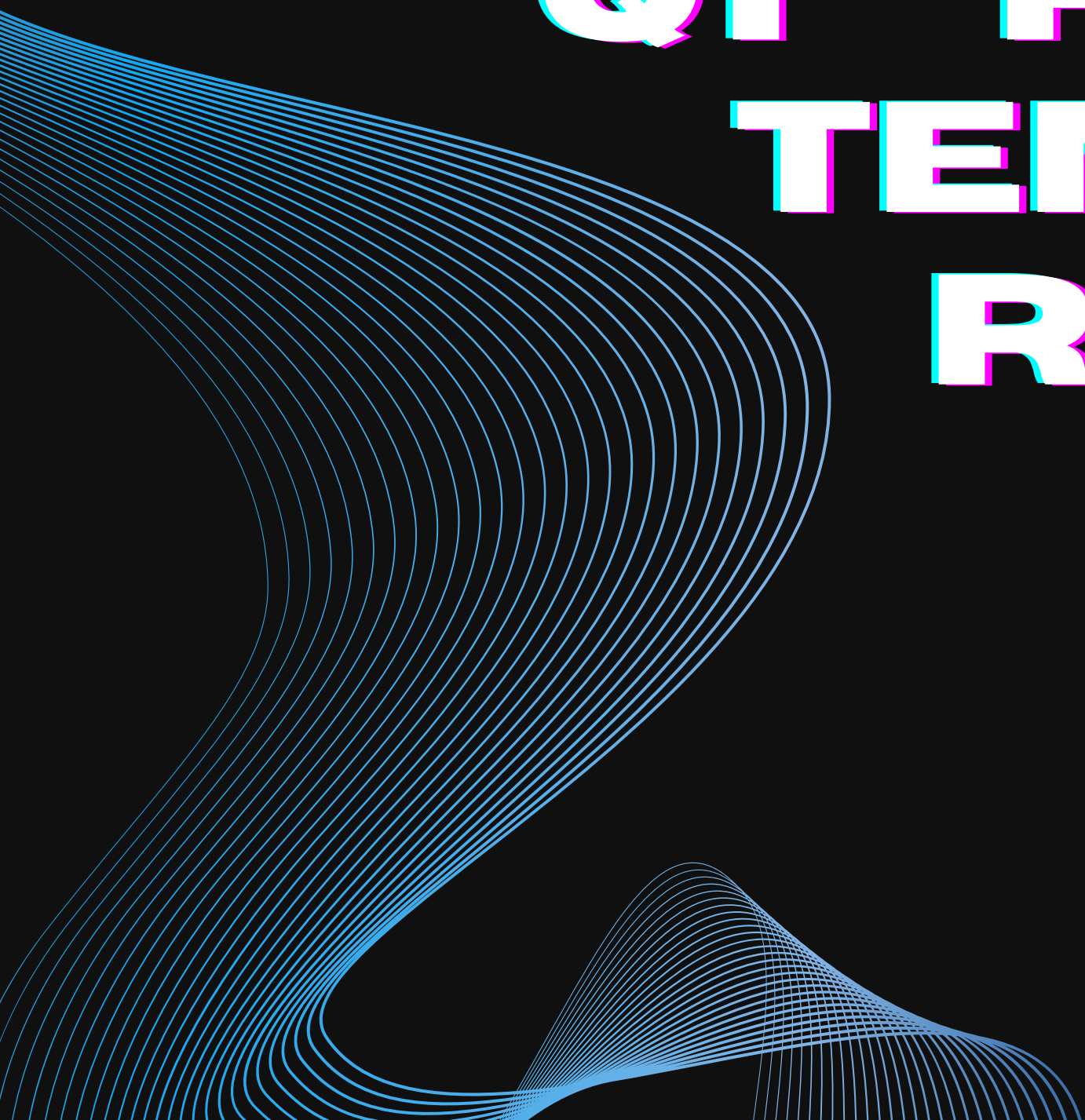


29 de Setembro de 2026

TRABALHO DE BANCO DE DADOS II

IFCE - INSTITUTO FEDERAL DO ESTADO DO CEARÁ, CAMPUS CRATO

An abstract graphic composed of numerous thin, light blue lines that curve and flow from the left side of the frame towards the center, creating a sense of motion and depth. The lines are more densely packed in some areas, forming a wave-like pattern.

Q1 – PANORAMA TEMPORALE RANKING

Funcionamento da consulta

- **dados_mensais** → reúne os dados através dos JOINS e agrupa por mês + categoria, calculando quantidade de locações e receita.
- **ranking_mensal** → utiliza RANK() OVER para classificar as categorias pela receita dentro de cada mês (PARTITION BY mes).

```
WITH dados_mensais AS (  
    SELECT  
        DATE_FORMAT(r.rental_date, '%Y-%m') AS mes,  
        c.name AS categoria,  
        COUNT(DISTINCT r.rental_id) AS qtd_locacoes,  
        SUM(p.amount) AS receita  
    FROM rental r  
    INNER JOIN inventory i ON r.inventory_id = i.inventory_id  
    INNER JOIN film f ON i.film_id = f.film_id  
    INNER JOIN film_category fc ON f.film_id = fc.film_id  
    INNER JOIN category c ON fc.category_id = c.category_id  
    INNER JOIN payment p ON r.rental_id = p.rental_id  
    GROUP BY  
        DATE_FORMAT(r.rental_date, '%Y-%m'),  
        c.name  
),  
ranking_mensal AS (  
    SELECT  
        mes,  
        categoria,  
        qtd_locacoes,  
        receita,  
        RANK() OVER (  
            PARTITION BY mes  
            ORDER BY receita DESC  
        ) AS ranking_no_mes  
    FROM dados_mensais  
),
```

- **Resultado** → utiliza SUM() OVER para calcular a receita acumulada de cada categoria ao longo dos meses.
- **Consulta final** → apresenta os dados, arredonda os valores e mantém apenas o Top 3 de cada mês.

Fluxo:

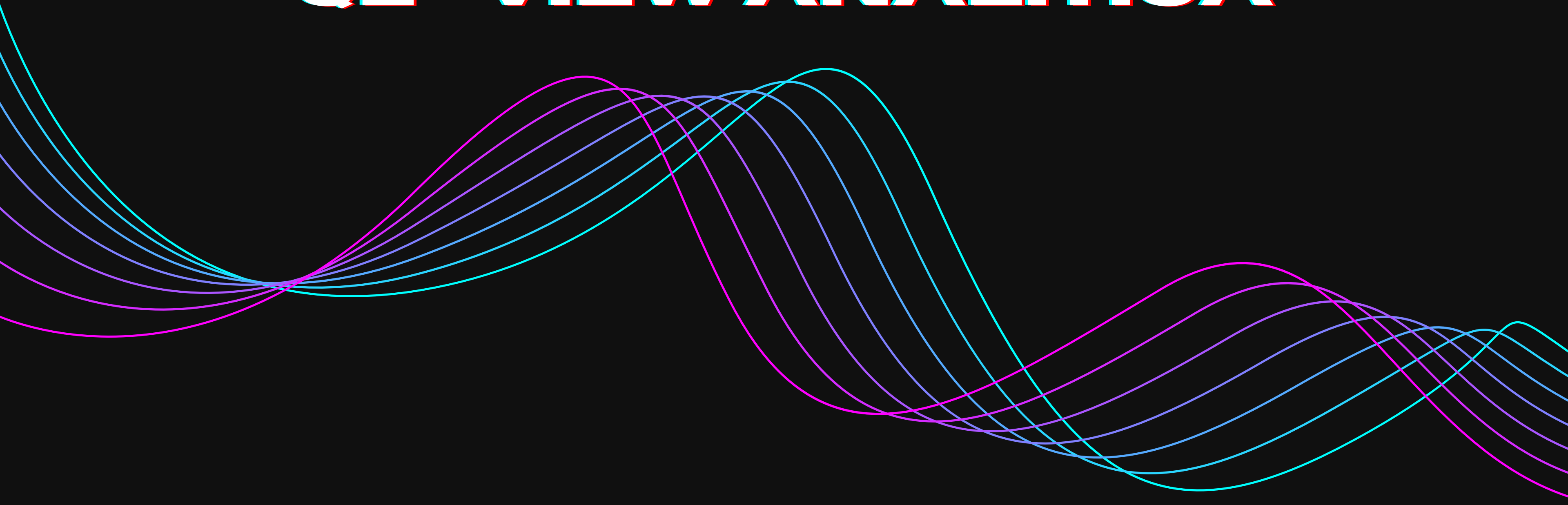
JOINS → Agrupamento → Ranking mensal → Acumulado → Top 3

```
resultado AS (  
    SELECT  
        mes,  
        categoria,  
        qtd_locacoes,  
        receita,  
        ranking_no_mes,  
        SUM(receita) OVER (  
            PARTITION BY categoria  
            ORDER BY mes  
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
        ) AS receita_acumulada  
    FROM ranking_mensal  
)  
SELECT  
    mes AS Mes,  
    categoria AS Categoria,  
    qtd_locacoes AS Qtd. locacoes,  
    ROUND(receita, 2) AS Receita,  
    ranking_no_mes AS Ranking no mes,  
    ROUND(receita_acumulada, 2) AS Receita acumulada  
FROM resultado  
WHERE ranking_no_mes <= 3  
ORDER BY  
    mes,  
    ranking_no_mes;
```

RESULTADO DA CONSULTA

Mes	Categoria	Qtd. locacoes	Receita	Ranking no mes	Receita acumulada
2005-05	Action	87	371.13	1	371.13
2005-05	Drama	85	370.15	2	370.15
2005-05	Documentary	86	353.14	3	353.14
2005-06	Sports	159	696.41	1	1029.65
2005-06	Animation	174	691.26	2	980.52
2005-06	Sci-Fi	162	685.38	3	1005.54
2005-07	Sports	497	2281.03	1	3310.68
2005-07	Sci-Fi	462	2013.38	2	3018.92
2005-07	Drama	463	1978.37	3	3031.00
2005-08	Sports	432	1962.68	1	5273.36
2005-08	Sci-Fi	385	1707.15	2	4726.07
2005-08	Animation	408	1654.92	3	4589.55
2006-02	Animation	21	66.75	1	4656.30
2006-02	Action	17	55.80	2	4375.85
2006-02	Sports	15	40.85	3	5314.21

Q2 - VIEW ANALÍTICA



- A regra de negócio é manter pelo menos 3 cópias disponíveis para aluguel de cada filme presente no inventário da loja.
- A view **filmes_criticos** reúne os dados apenas dos filmes atualmente alugados e com menos de 2 cópias restantes, assim formando uma lista de ocorrências. As colunas **qtd_alugado**, **qtd_total**, e **qtd_disponivel** mostram o estado atual de disponibilidade desse filme.
- **qtd_disponivel** é ao mesmo tempo o score de prioridade do filme, já que os filmes com menos cópias disponíveis devem ser reestocados com mais urgência.

```
WITH filmes_criticos AS (  
  > SELECT  
  >     film.film_id,  
  >     film.title,  
  >     film.replacement_cost,  
  >     COUNT(rental.rental_id) AS qtd_alugado,  
  >     COUNT(inventory.inventory_id) AS qtd_total,  
  >     COUNT(inventory.inventory_id) - COUNT(rental.rental_id) AS qtd_disponivel  
  > FROM sakila.film  
  > INNER JOIN sakila.inventory  
  >     ON inventory.film_id = film.film_id  
  > LEFT JOIN sakila.rental  
  >     ON rental.inventory_id = inventory.inventory_id AND rental.return_date IS NULL  
  > GROUP BY film.film_id, film.title, film.replacement_cost  
  > HAVING qtd_alugado != 0 AND qtd_disponivel <= 2  
  > )  
SELECT  
  > *,  
  > ROW_NUMBER() OVER (  
  >     ORDER BY filme.qtd_disponivel ASC, filme.replacement_cost ASC, filme.film_id ASC  
  > ) AS ordem_prioridade  
FROM filmes_criticos filme  
ORDER BY ordem_prioridade ASC;
```

- A query final cria uma ordem recomendada na qual estocar os filmes (**ordem_prioridade**), e apresenta os resultados nessa ordem.
- São priorizados os filmes com menos cópias disponíveis, depois os filmes mais baratos, e por último, os IDs dos filmes são usados para eliminar empates. Então, a ordem recomendada é determinística, e é usado ROW_NUMBER no lugar de RANK.

```
WITH filmes_criticos AS (  
  > SELECT  
  >   film.film_id,  
  >   film.title,  
  >   film.replacement_cost,  
  >   COUNT(rental.rental_id) AS qtd_alugado,  
  >   COUNT(inventory.inventory_id) AS qtd_total,  
  >   COUNT(inventory.inventory_id) - COUNT(rental.rental_id) AS qtd_disponivel  
  > FROM sakila.film  
  > INNER JOIN sakila.inventory  
  >   ON inventory.film_id = film.film_id  
  > LEFT JOIN sakila.rental  
  >   ON rental.inventory_id = inventory.inventory_id AND rental.return_date IS NULL  
  > GROUP BY film.film_id, film.title, film.replacement_cost  
  > HAVING qtd_alugado != 0 AND qtd_disponivel <= 2  
  > )  
SELECT  
  > *,  
  > ROW_NUMBER() OVER (  
  >   ORDER BY filme.qtd_disponivel ASC, filme.replacement_cost ASC, filme.film_id ASC  
  > ) AS ordem_prioridade  
FROM filmes_criticos filme  
ORDER BY ordem_prioridade ASC;
```

RESULTADO DA CONSULTA:

film_id	title	replacement_cost	qtd_alugado	qtd_total	qtd_disponivel	ordem_prioridade
998	ZHIVAGO CORE	10.99	1	2	1	1
264	DWARFS ALTER	13.99	1	2	1	2
405	HAUNTED ANTITRUST	13.99	1	2	1	3
605	MULHOLLAND BEAST	13.99	1	2	1	4
990	WORLD LEATHERNECKS	13.99	1	2	1	5
375	GRAIL FRANKENSTEIN	17.99	1	2	1	6
935	VANISHED GARDEN	17.99	1	2	1	7
102	BUBBLE GROSSE	20.99	1	2	1	8
399	HAPPINESS UNITED	23.99	1	2	1	9
974	WILD APOLLO	24.99	1	2	1	10
53	BANG KWAI	25.99	1	2	1	11
178	CONNECTION MICROCOSMOS	25.99	1	2	1	12
639	OPPOSITE NECKLACE	9.99	1	3	2	13
604	MULAN MOON	10.99	1	3	2	14
2	ACE GOLDFINGER	12.99	1	3	2	15
792	SHREK LICENSE	15.99	1	3	2	16
581	MINORITY KISS	16.99	1	3	2	17
5	AFRICAN EGG	22.99	1	3	2	18
205	DANCES NONE	22.99	2	4	2	19
338	FRISCO FORREST	23.99	1	3	2	20
146	CHITTY LOCK	24.99	1	3	2	21
977	WINDOW SIDE	25.99	1	3	2	22
530	LORD ARIZONA	27.99	1	3	2	23
819	SONG HEDWIG	29.99	1	3	2	24

24 rows in set (0.017 sec)



Q3 – GARANTIA DE REGRAS DE NEGÓCIO

Regra de negócio :

Garantir que pagamento devem ter um valor mínimo maior que 0

Fluxo:

Validação → Valor inválido? → Registro do Log → SIGNAL → Pagamento bloqueado → Consulta da Auditoria

tb_log_auditoria → cria a tabela responsável por armazenar os registros de tentativas de inserção de dados inválidos na base de dados.

```
722
723 --codigo
724 CREATE TABLE IF NOT EXISTS tb_log_auditoria (
725     id_log INT AUTO_INCREMENT PRIMARY KEY,
726     data_hora DATETIME DEFAULT CURRENT_TIMESTAMP,
727     usuario VARCHAR(100) NOT NULL,
728     operacao VARCHAR(20) NOT NULL,
729     tabela_afetada VARCHAR(50) NOT NULL,
730     identificacao_problema TEXT NOT NULL
731 ) ENGINE = MyISAM;
732
733
734 ALTER TABLE tb_log_auditoria ENGINE = MyISAM;
735
736
737 DELIMITER //
738
739 CREATE TRIGGER trg_valida_pagamento_bi
740 BEFORE INSERT ON payment
741 FOR EACH ROW
742 BEGIN
743
744     IF NEW.amount <= 0 THEN
745         INSERT INTO tb_log_auditoria (
746             usuario,
747             operacao,
748             tabela_afetada,
749             identificacao_problema
750         )
751         VALUES (
752             USER(),
753             'INSERT',
754             'payment',
755             CONCAT(
756                 'Tentativa de inserção de pagamento inválida de R$ ',
757                 NEW.amount,
758                 ' para o cliente ID ',
759                 NEW.customer_id
760             )
761         );
762         SIGNAL SQLSTATE '45000'
763         SET MESSAGE_TEXT =
764             'VIOLAÇÃO DE REGRA DE NEGÓCIO: O valor do pagamento deve ser estritamente maior que R$ 0,00.';
765     END IF;
766 END//
767
768 DELIMITER ;
769
```

- **ENGINE = MyISAM** → permite que o registro de auditoria permaneça gravado mesmo quando a inserção do pagamento for interrompida pelo **SIGNAL**.
- **trg_valida_pagamento_bi** → executado antes de uma inserção na tabela payment e verifica se o valor do pagamento (**NEW.amount**) é maior que zero.
- posteriormente as tentativas de inserção inválidas registradas no banco.
- **INSERT INTO tb_log_auditoria** → quando o valor é menor ou igual a zero, registra o usuário, a operação, a tabela afetada e a descrição do problema.
- **SIGNAL** → interrompe a inserção do pagamento e apresenta uma mensagem personalizada informando a violação da regra de negócio.
- **Consulta à auditoria** → permite verificar posteriormente as tentativas de inserção inválidas registradas no banco.

```

733
734 ALTER TABLE tb_log_auditoria ENGINE = MyISAM;
735
736
737 DELIMITER //
738
739 CREATE TRIGGER trg_valida_pagamento_bi
740 BEFORE INSERT ON payment
741 FOR EACH ROW
742 BEGIN
743
744     IF NEW.amount <= 0 THEN
745         INSERT INTO tb_log_auditoria (
746             usuario,
747             operacao,
748             tabela_afetada,
749             identificacao_problema
750         )
751         VALUES (
752             USER(),
753             'INSERT',
754             'payment',
755             CONCAT(
756                 'Tentativa de inserção de pagamento inválido de R$ ',
757                 NEW.amount,
758                 ' para o cliente ID ',
759                 NEW.customer_id
760             )
761         );
762         SIGNAL SQLSTATE '45000'
763         SET MESSAGE_TEXT =
764             'VIOLAÇÃO DE REGRA DE NEGÓCIO: O valor do pagamento deve ser estritamente maior que R$ 0,00.';
765     END IF;
766 END//
767
768 DELIMITER ;
769

```

```
64 • INSERT INTO payment (  
65     customer_id,  
66     staff_id,  
67     rental_id,  
68     amount,  
69     payment_date )  
70     VALUES ( 1, 1, 1, 0.00, NOW() );  
71  
72  
73 • SELECT * FROM tb_log_auditoria;  
74
```

Output

Action Output

#	Time	Action	Message
60	15:09:55	CREATE TRIGGER trg_valida_pagamento_bi BEFORE INSERT ON payment FOR EACH ROW BEGIN	Error Code: 1359, Trigger already exists
61	15:10:37	DROP TRIGGER IF EXISTS trg_valida_pagamento_bi	0 row(s) affected
62	15:10:45	CREATE TRIGGER trg_valida_pagamento_bi BEFORE INSERT ON payment FOR EACH ROW BEGIN	0 row(s) affected
63	15:11:01	INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date) VALUES (1, ...	Error Code: 1644, VIOLAÇÃO DE REGRA DE NEGÓCIO: O valor do pagamento deve ser estritamente maior q
64	15:11:33	SELECT * FROM tb_log_auditoria LIMIT 0, 1000	1 row(s) returned
65	15:45:41	INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date) VALUES (1, 1, 1, 0.00, ...	Error Code: 1644, VIOLAÇÃO DE REGRA DE NEGÓCIO: O valor do pagamento deve ser estritamente maior q

INSERT INTO payment → exemplo de tentativa de pagamento não autorizado resultando em um “erro” que será salvo na tabela de auditoria

71

72

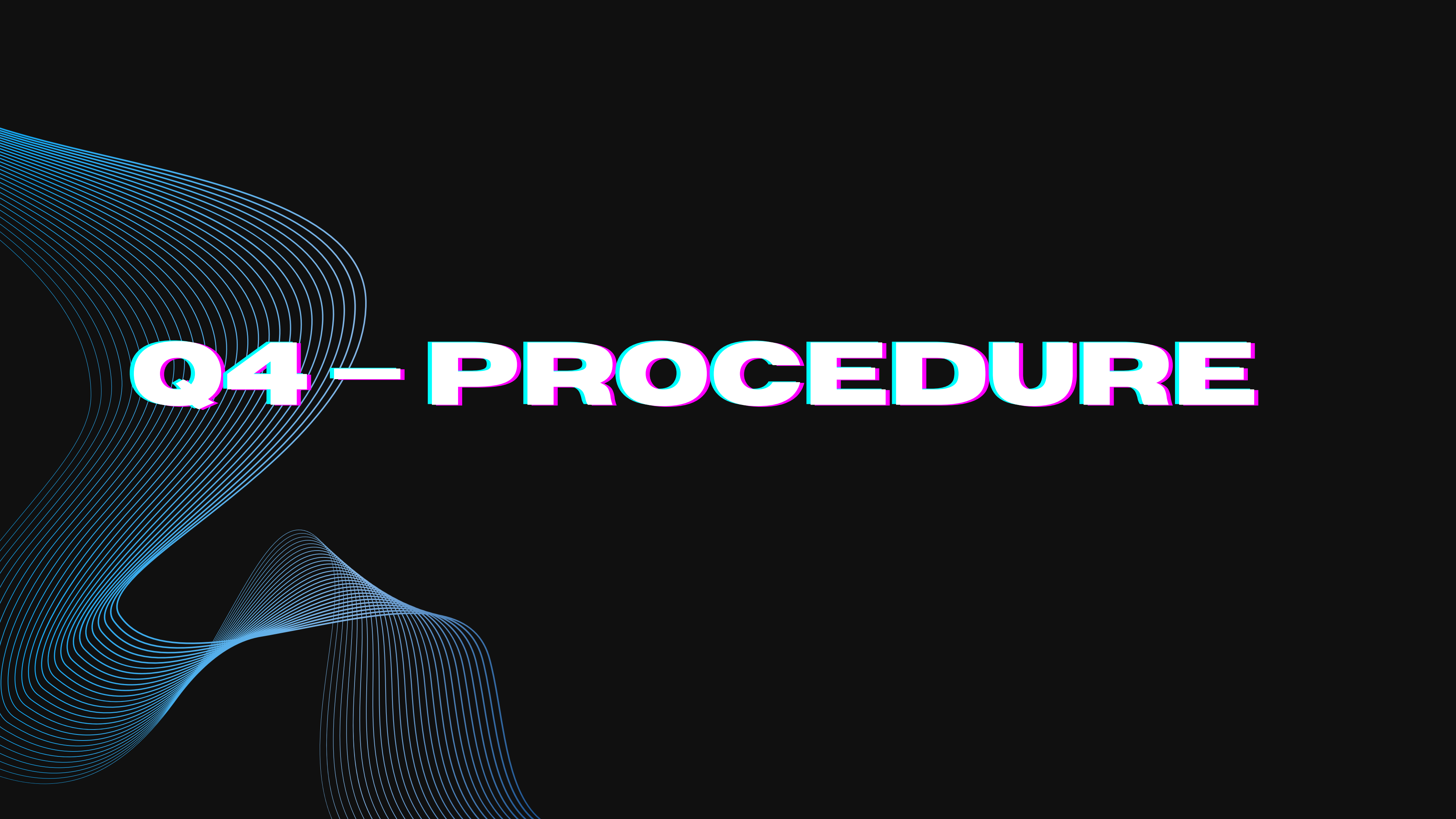
73 • `SELECT * FROM tb_log_auditoria;`

74

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	id_log	data_hora	usuario	operacao	tabela_afetada	identificacao_problema
▶	2	2026-09-29 15:11:01	root@localhost	INSERT	payment	Tentativa de inserção de pagamento inválido de...
	3	2026-09-29 15:45:41	root@localhost	INSERT	payment	Tentativa de inserção de pagamento inválido de...

SELECT da tb_log_auditoria → mostra de relatório de tentativas de pagamento errôneos



Q4 – PROCEDURE

CRIAÇÃO DA PROCEDURE

- Recebe 4 parâmetros opcionais: data início, data fim, categoria e loja
- Se algum parâmetro vier NULL, assume um valor padrão
- Valida as entradas antes de rodar a consulta (data inicial maior que final, categoria ou loja que não existem) e bloqueia se algo estiver errado
- Executa a consulta com os filtros aplicados e devolve uma tabela como resultado

```
DROP PROCEDURE IF EXISTS sp_relatorio_receita_categoria;

DELIMITER $$

CREATE PROCEDURE sp_relatorio_receita_categoria(
    IN p_data_inicio DATE,
    IN p_data_fim DATE,
    IN p_categoria VARCHAR(25),
    IN p_loja TINYINT
)
BEGIN
    IF p_data_inicio IS NULL THEN
        SET p_data_inicio = (SELECT DATE(MIN(rental_date)) FROM rental);
    END IF;
    IF p_data_fim IS NULL THEN
        SET p_data_fim = (SELECT DATE(MAX(rental_date)) FROM rental);
    END IF;

    IF p_data_inicio > p_data_fim THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Data inicial não pode ser maior que a data final.';
    END IF;

    IF p_categoria IS NOT NULL
        AND NOT EXISTS (SELECT 1 FROM category WHERE name = p_categoria) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Categoria informada não existe.';
    END IF;

    IF p_loja IS NOT NULL
        AND NOT EXISTS (SELECT 1 FROM store WHERE store_id = p_loja) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loja informada não existe.';
    END IF;
```

Q4 — Relatório da Procedure

- Receita e locações agrupadas por categoria de filme e por loja
- Total de locações de cada categoria
- Receita total gerada por categoria
- Ticket médio (dividida pela quantidade de pagamentos)
- Ranking de cada categoria por receita, calculado separadamente dentro de cada loja

```
WITH base AS (  
    SELECT c.name AS categoria, i.store_id AS loja, r.rental_id, p.amount  
    FROM rental r  
    JOIN payment p  
        ON p.rental_id = r.rental_id  
    JOIN inventory i  
        ON i.inventory_id = r.inventory_id  
    JOIN film_category fc  
        ON fc.film_id = i.film_id  
    JOIN category c  
        ON c.category_id = fc.category_id  
  
    WHERE r.rental_date >= p_data_inicio  
    AND r.rental_date < DATE_ADD(p_data_fim, INTERVAL 1 DAY)  
    AND (p_categoria IS NULL OR c.name = p_categoria)  
    AND (p_loja IS NULL OR i.store_id = p_loja)  
)  
  
SELECT categoria AS Categoria,  
    loja AS Loja,  
    COUNT(DISTINCT rental_id) AS Total_Locacoes,  
    SUM(amount) AS Receita_Total,  
    ROUND(AVG(amount), 2) AS Ticket_Medio,  
    RANK() OVER (PARTITION BY loja ORDER BY SUM(amount) DESC) AS Ranking_na_Loja  
FROM base  
GROUP BY categoria, loja  
ORDER BY loja, Ranking_na_Loja;  
END$$  
  
DELIMITER ;
```

RESULTADO DA CONSULTA:

CALL sp_relatorio_receita_categoria('2005-07-01', '2005-07-31', NULL, NULL);

mes_referencia	categoria	total_locacoes	clientes_unicos	filmes_distintos	receita_total	ticket_medio
2005-05	Action	87	81	50	371.13	4.27
2005-05	Animation	74	68	49	289.26	3.91
2005-05	Children	71	66	40	288.29	4.06
2005-05	Classics	62	57	40	227.38	3.67
2005-05	Comedy	72	65	38	295.28	4.10
2005-05	Documentary	86	79	51	353.14	4.11
2005-05	Drama	85	80	51	370.15	4.35
2005-05	Family	85	83	52	304.15	3.58
2005-05	Foreign	71	65	46	296.29	4.17
2005-05	Games	69	65	39	293.31	4.25
2005-05	Horror	53	50	30	243.47	4.59
2005-05	Music	61	58	36	268.39	4.40
2005-05	New	60	58	36	302.40	5.04
2005-05	Sci-Fi	84	81	46	320.16	3.81
2005-05	Sports	76	74	49	333.24	4.38

15 rows in set (0.135 sec)

Q5 – VIEWS PARA DIFERENTES PERFIS



Perfil 1 — Gerencial

A View gerencial apresenta uma visão mais geral da locadora.

Os dados são organizados por mês e categoria, mostrando:

- Total de locações
- Clientes únicos
- Filmes distintos
- Receita total
- Ticket médio

```
CREATE VIEW vw_painel_filmes_gerencial AS
SELECT
    DATE_FORMAT(r.rental_date, '%Y-%m') AS mes_referencia,
    cat.name AS categoria,
    COUNT(r.rental_id) AS total_locacoes,
    COUNT(DISTINCT r.customer_id) AS clientes_unicos,
    COUNT(DISTINCT f.film_id) AS filmes_distintos,
    ROUND(SUM(p.amount), 2) AS receita_total,
    ROUND(AVG(p.amount), 2) AS ticket_medio
FROM rental r
JOIN inventory i
    ON r.inventory_id = i.inventory_id
JOIN film f
    ON i.film_id = f.film_id
JOIN film_category fc
    ON f.film_id = fc.film_id
JOIN category cat
    ON fc.category_id = cat.category_id
LEFT JOIN payment p
    ON r.rental_id = p.rental_id
GROUP BY
    DATE_FORMAT(r.rental_date, '%Y-%m'),
    cat.category_id,
    cat.name;
```

RESULTADO DA CONSULTA

mes_referencia	categoria	total_locacoes	clientes_unicos	filmes_distintos	receita_total	ticket_medio
2005-05	Action	87	81	50	371.13	4.27
2005-05	Animation	74	68	49	289.26	3.91
2005-05	Children	71	66	40	288.29	4.06
2005-05	Classics	62	57	40	227.38	3.67
2005-05	Comedy	72	65	38	295.28	4.10
2005-05	Documentary	86	79	51	353.14	4.11
2005-05	Drama	85	80	51	370.15	4.35
2005-05	Family	85	83	52	304.15	3.58
2005-05	Foreign	71	65	46	296.29	4.17
2005-05	Games	69	65	39	293.31	4.25
2005-05	Horror	53	50	30	243.47	4.59
2005-05	Music	61	58	36	268.39	4.40
2005-05	New	60	58	36	302.40	5.04
2005-05	Sci-Fi	84	81	46	320.16	3.81
2005-05	Sports	76	74	49	333.24	4.38
2005-05	Travel	60	59	33	267.40	4.46
2005-06	Action	160	143	59	681.40	4.26
2005-06	Animation	174	148	63	691.26	3.97
2005-06	Children	130	118	53	495.70	3.81
2005-06	Classics	136	122	52	518.64	3.81
2005-06	Comedy	135	124	52	639.65	4.74
2005-06	Documentary	160	141	60	612.40	3.83
2005-06	Drama	152	135	59	682.48	4.49
2005-06	Family	154	137	64	577.46	3.75
2005-06	Foreign	142	130	63	585.58	4.12
2005-06	Games	142	123	54	628.58	4.43
2005-06	Horror	116	105	46	508.84	4.39
2005-06	Music	133	115	48	526.67	3.96
2005-06	New	132	117	55	565.68	4.29
2005-06	Sci-Fi	162	147	59	685.38	4.23
2005-06	Sports	159	139	68	696.41	4.38
2005-06	Travel	124	110	45	533.76	4.30
2005-07	Action	464	318	61	1804.36	3.89
2005-07	Animation	489	325	64	1954.11	4.00
2005-07	Children	406	303	58	1539.94	3.79
2005-07	Classics	384	290	54	1497.16	3.90
2005-07	Comedy	383	284	56	1829.17	4.78
2005-07	Documentary	429	294	63	1726.71	4.02
2005-07	Drama	463	318	61	1978.37	4.27

Perfil 2 — Analítico

A View analítica apresenta uma visão mais detalhada da locadora. Os dados são organizados por transação individual, mostrando:

- ID e data da operação
- Dados do cliente (nome e e-mail)
- Filme alugado e categoria
- Loja onde foi realizada
- Valor pago na transação

```
CREATE VIEW vw_painel_filmes_analitico AS
SELECT
    r.rental_id AS id_locacao,
    r.rental_date AS data_locacao,
    r.return_date AS data_devolucao,
    f.title AS filme,
    cat.name AS categoria,
    CONCAT(c.first_name, ' ', c.last_name) AS cliente,
    c.email AS email_cliente,
    ci.city AS cidade,
    p.amount AS valor_pago
FROM rental r
JOIN inventory i
    ON r.inventory_id = i.inventory_id
JOIN film f
    ON i.film_id = f.film_id
JOIN film_category fc
    ON f.film_id = fc.film_id
JOIN category cat
    ON fc.category_id = cat.category_id
JOIN customer c
    ON r.customer_id = c.customer_id
JOIN address a
    ON c.address_id = a.address_id
JOIN city ci
    ON a.city_id = ci.city_id
LEFT JOIN payment p
    ON r.rental_id = p.rental_id;
```

RESULTADO DA CONSULTA

	MariaDB [sakila]> +-----+-----+-----+-----+-----+-----+-----+-----+
	+-----+-----+-----+-----+-----+-----+-----+-----+
	-> id_locacao data_locacao data_devolucao filme categoria cliente email_cliente cidade
	valor_pago
	-> +-----+-----+-----+-----+-----+-----+-----+-----+
	+-----+-----+-----+-----+-----+-----+-----+-----+
	-> 10895 2005-08-02 01:16:59 2005-08-03 02:41:59 AMADEUS HOLY Action JANE BENNETT JANE.BENNETT@sakilacustomer.org Aratuba
	0.99
	-> 12268 2005-08-18 04:26:54 2005-08-23 06:40:54 AMADEUS HOLY Action DEBRA NELSON DEBRA.NELSON@sakilacustomer.org Vila Velha
	0.99
	-> 3150 2005-06-20 20:35:28 2005-06-26 01:01:28 AMADEUS HOLY Action REBECCA SCOTT REBECCA.SCOTT@sakilacustomer.org Kurashiki
	0.99
	-> 5072 2005-07-09 05:01:58 2005-07-18 08:17:58 AMADEUS HOLY Action MAXINE SILVA MAXINE.SILVA@sakilacustomer.org Poços de Caldas
	3.99
	-> 9080 2005-07-30 08:02:39 2005-08-06 12:02:39 AMADEUS HOLY Action BRANDY GRAVES BRANDY.GRAVES@sakilacustomer.org Warren
	1.99
	-> 14603 2005-08-21 17:51:06 2005-08-28 15:36:06 AMADEUS HOLY Action JESSICA HALL JESSICA.HALL@sakilacustomer.org Città del Vaticano
	1.99
ano	-> 588 2005-05-28 12:08:37 2005-05-29 16:25:37 AMADEUS HOLY Action JULIA FLORES JULIA.FLORES@sakilacustomer.org La Plata
	0.99
	-> 2867 2005-06-20 00:08:38 2005-06-23 19:35:38 AMADEUS HOLY Action VERNON CHAPA VERNON.CHAPA@sakilacustomer.org Denizli
	0.99
	-> 6030 2005-07-11 02:37:51 2005-07-16 22:34:51 AMADEUS HOLY Action KATHLEEN ADAMS KATHLEEN.ADAMS@sakilacustomer.org Arak
	0.99
	-> 7788 2005-07-28 07:21:55 2005-08-03 06:33:55 AMADEUS HOLY Action HECTOR POINDEXTER HECTOR.POINDEXTER@sakilacustomer.org Pjatigorsk
	0.99
	-> 14310 2005-08-21 07:44:32 2005-08-22 09:02:32 AMADEUS HOLY Action SAMANTHA DUNCAN SAMANTHA.DUNCAN@sakilacustomer.org Matamoros
	0.99
	-> 10509 2005-08-01 11:25:28 2005-08-10 09:19:28 AMADEUS HOLY Action NATHANIEL ADAM NATHANIEL.ADAM@sakilacustomer.org Joliet
	3.99
	-> 14349 2005-08-21 08:54:53 2005-08-30 14:27:53 AMADEUS HOLY Action SHERRY MARSHALL SHERRY.MARSHALL@sakilacustomer.org Shubra al-Khayma
	3.99
	-> 1735 2005-06-16 15:51:52 2005-06-20 13:27:52 AMADEUS HOLY Action BARBARA JONES BARBARA.JONES@sakilacustomer.org Myingyan
	0.99
	-> 3474 2005-07-05 22:59:53 2005-07-08 20:32:53 AMADEUS HOLY Action EDUARDO HIATT EDUARDO.HIATT@sakilacustomer.org Jining
	0.99
	-> +-----+-----+-----+-----+-----+-----+-----+-----+
	+-----+-----+-----+-----+-----+-----+-----+-----+

**OBRIGADO POR
ASSISTIR!**

